

コンピュータアーキテクチャI

担当：佐々木 敬泰

第5回

基本命令

命令の分類

- 機能による分類
 - 算術演算
 - 加減乗除、剰余等
 - 論理演算
 - ビット和・積・排他的論理和・シフト等
 - 定数ロード命令
 - メモリアクセス命令
 - メモリから読み出し、メモリへの書き込み
 - 分岐命令
 - 条件分岐
 - 無条件分岐
 - 疑似命令
 - 1つ以上の命令の組み合わせで実現する

3

算術演算命令

コンピュータアーキテクチャI

5

例題

- 以下のプログラムに対応するアセンブラプログラムを考えよ

```
int func(int a, int b, int c, int d){  
    return (a + b) - (c + d);  
}
```

この部分

- ヒント
 - 関数の引数：\$4, \$5, \$6, \$7
 - 関数の戻り値：\$2
 - $\$2 = (\$4 + \$5) - (\$6 + \$7)$

- 解答

コンピュータアーキテクチャI

7

シラバス

1. イントロダクション
2. コンピュータの構成
3. 数の表現
4. 様々なプロセッサと命令セット
5. 基本命令
6. メモリアクセス命令
7. 分岐命令
8. 高級言語からアセンブラへの変換
9. プログラムの翻訳と起動
10. 加減算
11. 乗算アルゴリズムと乗算器
12. 除算アルゴリズムと除算器
13. 浮動小数点
14. 浮動小数点の乗算と誤差
15. 定期試験

コンピュータアーキテクチャI

2

今日の内容

- 算術演算命令
- 論理演算命令
- 定数ロード命令
- 疑似命令

コンピュータアーキテクチャI

4

算術演算

- add \$1, \$2, \$3 (add)
 - $\$1 = \$2 + \$3$
- sub \$1, \$2, \$3 (subtract)
 - $\$1 = \$2 - \$3$
- addi \$1, \$2, 12 (add immediate)
 - $\$1 = \$2 + 12$
 - 定数（即値）は符号付き16bit整数
- subiはないの???
 - addiは符号付き整数を取るためaddiで代用
 $\text{subi } \$1, \$2, 10 \Rightarrow \text{addi } \$1, \$2, -10$

6

符号付き加算と符号なし加算

- 2の補数表現を取っているコンピュータでは、符号なしの加算と符号ありの加算結果は同じ。
- しかし、MIPSには、符号ありの加算命令(ADD)と符号なしの加算命令(ADDU)がある。何故か？
 - 4bitレジスタで考える
 - 符号あり： $(-3) + (-4) = -7$
 - $1101_2 + 1100_2 = 1001_2$
 - 符号なし： $13 + 12 = 9$
 - $1101_2 + 1100_2 = 1001_2$
 - 1101_2 を符号ありと解釈するか符号なしと解釈するかの差では、ADDとADDUの差は？

コンピュータアーキテクチャI

8

ADDとADDUの違い

- ADD命令とADDU命令
 - 演算結果は同じ
 - ADDはオーバーフロー時に例外を起こす
 - ADDUは常に例外を起こさない

【発展】
C言語は整数加減算のオーバーフローを無視する。そのため、
int a, b, c;
unsigned d, e, f;
a = b + c;
d = e + f;
は、どちらもaddu命令に翻訳される。

乗算命令(mult/multu)

- Nbit同士の乗算結果は2Nbit
- 32bitの乗算は64bitになる
 - ⇒乗算結果がレジスタに収まらない
 - 1. あふれた桁は破棄
 - 2. 専用のレジスタに結果を格納 ⇐ MIPS
- 乗算専用レジスタ

- HIレジスタ、LOレジスタ

HIレジスタ	LOレジスタ
上位32bit	下位32bit

- mfhi(move from hi)、mflo(move from lo)命令を使って汎用レジスタに値を取り込む

乗算の例

- コード例
 - 1命令だが実行には複数サイクル掛かる
 - mult \$1, \$2 # \$1 × \$2
 - mfhi \$1 # \$1は積の上位32bit
 - mflo \$2 # \$2は積の下位32bit
- 符号の有無
 - mult命令は符号付き乗算
 - multu命令は符号なし乗算
- オーバーフロー
 - オーバーフローは無視される
 - プログラムで解決する

除算・剰余

- 除算も専用レジスタを用いる
 - 32bitの除算なら商は32bitで収まるのでは？
⇒除算命令は同時に剰余（余り）も求める
理由は後の回で説明
- 除算命令
 - div \$1, \$2 # 符号付き除算
 - divu \$1, \$2 # 符号なし除算
 - LOレジスタに商、HIレジスタに剰余が格納
- オーバーフロー/ゼロ除算
 - ハードウェア的には無視
 - ゼロ除算の検出等が必要ならソフトウェアで実装

具体例

- \$2 = (\$4 * \$5) / \$6を考える
 - mult \$4, \$5 # {HI, LO} = \$4 * \$5
 - mflo \$2 # \$2 = LO
 - div \$2, \$6 # LO = \$2 / \$6 (HI = \$2 % \$6)
 - mflo \$2 # \$2 = LO

【発展】
乗算を1回実行するのに、mult命令とmflo命令を実行するのは非効率なので、MIPS32命令セットからはMULという新しい命令が加わった。
MUL \$2, \$4, \$5 # \$2 = \$4 * \$5

論理演算（ビット毎の和積等）

- ビット和
 - OR \$1, \$2, \$3
 - ORI \$1, \$2, Imm
Immは符号なし16bit即値
- ビット積
 - AND \$1, \$2, \$3
 - ANDI \$1, \$2, Imm
- 否定
 - NOT命令はない
 - NOR(Not OR)で代用する
 - NOR \$1, \$1, \$0 # \$1 = ~(\$1 | \$0)

ゼロレジスタ

論理演算命令

ビット積（和）と論理積（和）

- 積（和）演算には、
 - ビット毎
 - 論理的の二種類の演算がある。
- C言語例)
a=0100₂、b=0010₂
• ビット毎の積（AND命令）
if(a & b) ⇒ a&b = 0000₂なので偽
• 論理的な積（複数命令で実現）
if(a && b) ⇒ a && b = 0001₂なので真
⇒後の回で説明予定

シフト演算

- ビット単位でデータを左右に移動
 - 右シフト
 - $0010_2 \Rightarrow 0001_2$
 - 左シフト
 - $0010_2 \Rightarrow 0100_2$
- MIPS命令セット
 - 左に4bitシフト (shift left logical)
 - `sll $1, $2, 4` # \$1 = \$2 << 4
 - 右シフトはsrl(shift right logical)命令

17

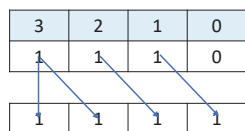
シフト命令による値の変化

- 左シフト命令
 - $0010_2(2)$ を左シフト $0100_2(4)$
 - $0110_2(6)$ を左シフト $1100_2(12)$
 $\Rightarrow 1\text{bit}$ 左シフトで値が**倍**になる
 $\Rightarrow N\text{ビット}$ 左シフトで **2^N 倍**になる
- 右シフト命令と値の変化
 - $0010_2(2)$ を右シフト $0001_2(1)$
 - $0110_2(6)$ を右シフト $0011_2(3)$
 $\Rightarrow 1\text{bit}$ 右シフトで値が**半分**になる
 $\Rightarrow N\text{ビット}$ 右シフトで **2^N 分の1**になる

18

論理シフトと算術シフト

- 4bitの符号付き整数を考える
 - $1110_2(-2)$ を右シフトすると $0111(7)$ になる
 - 2を右シフトしたら-1 (1/2倍) になって欲しい
 - sra (shift right arithmetic) 命令 (算術右シフト命令)
 - `sra $1, $2, 3`



算術左シフト命令は？

sllはsllと同じ結果になるので存在しない

19

例題

- \$4の値が $12345678_{16}(305419896_{10})$, \$5の値が $87654321_{16}(2271560481_{10})$ だったとする。
- 以下の命令を実行後の\$10～\$15の値を考えよ。

- `sll $10, $4, 4`
- `srl $11, $4, 4`
- `sra $12, $4, 4`
- `sll $13, $5, 4`
- `srl $14, $5, 4`
- `sra $15, $5, 4`

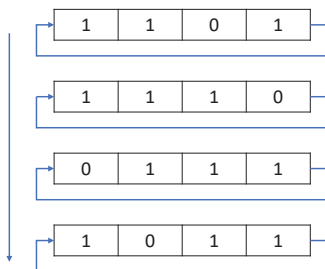
補足

コンピュータアーキテクチャI

20

ビットローテート命令

- シフト命令は追い出されたビットは消える



ローテート命令

x86/x64等の一部のプロセッサでは実装されているが、すべてのプロセッサに存在する訳ではない。

コンピュータアーキテクチャI

21

レジスタへの定数代入

- 例) \$1に0x12345678を書き込む
 - MIPSの機械語は32bit
 - 32bit即値代入命令を32bit以内で作るのは不可！
 $\Rightarrow$ 複数の命令を組み合わせる
 - LUI (load upper immediate)命令
 - `lui $1, 0x1234` # \$1 = (0x1234 << 16)
- | 上位16bit | 下位16bit |
|---------|---------|
| 0x1234 | 0x0000 |
- 実装例
 - `lui $1, 0x1234`
 - `ori $1, $1, 0x5678`

23

定数ロード命令

コンピュータアーキテクチャI

22

疑似命令

コンピュータアーキテクチャI

24

疑似命令

- アセンブラは基本的に機械語ご1対1対応
 - 機械語を直接読み書きするよりはマシ
 - 人間にはやや不便
- 人間にとって使いやすくした（仮想）命令
 - アセンブラが1つ以上の命令に変換
- 例)
 - \$1の値を\$2に移動（コピー）
 - `add $2, $0, $1` $\# \$2 = \$0 + \$1 \Rightarrow \$2 = \$1$
 $\Rightarrow$ `move $2, $1`

疑似命令の例

- 16bit即値の代入：li (load immediate)
`li $2, 0x1234`
↓
`addiu $2, $0, 0x1234`
- 32bit即値の代入：li (load immediate)
`li $2, 0x12345678`
↓
`lui $2, 0x1234`
`ori $2, $2, 0x5678`
- 何もしない命令：nop (no operation)
`nop`
↓
`sll $0, $0, 0`

x86/x64 プロセッサ

- RISCプロセッサの多くは3オペランド(書き込みレジスタが1つと読み込みレジスタが2つ)なので、`add`命令や`or`命令とゼロレジスタの組合せで`move`命令を代用できる
 - $\$1 \Rightarrow \2
 - `add $2, $1, $0`
 - `or $2, $1, $0`
- Intel x86/x64系プロセッサは基本的に2オペランドなので、機械語として`move`命令が存在

疑似命令の例（続き）

- 即値の引き算：subi (subtract immediate)
`subi $2, $3, 30`
↓
`addi $2, $3, -30`
- 他にもいろいろある